

Module 7

Coverage and Assertions

Learning objectives

- Master basic coverage analysis and assertion-based verification without SystemVerilog dependencies

Prerequisites

- See module README

Learning path

Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["Coverage Fundamentals"]

T2["Code Coverage"]

T1 --> T2

T3["Functional Coverage (Manual ")

T2 --> T3

Master basic coverage analysis and assertion-based verification without SystemVerilog dependencies

Overview

- This module covers fundamental coverage analysis and assertion-based verification using basic Veril...

Design architecture

1. Observability-focused DUT set

- ``dut/simple_gates/``,
 ``dut/multiplexers/``,
 ``dut/counters/``: Familiar blocks for assertion and coverage...
- Checkpoints: Internal signals (count, select, flags) exposed for bind-style or inline assertions
- Tool split: Assertions in SV/Verilog for iverilog; C++ ``assert`` paths in Verilator examples

Rtl Architecture

```
Diagram: Rtl Architecture
(render with mmdc when available)
flowchart TB
  DUT[counter_4bit] --> IMM[immediate checks]
  DUT --> CLKAST[clocked assertions]
  IMM --> LOG[assert pass/fail count]
  CLKAST --> LOG
  LIB[assertion library macros] --> CLKAST
```

2. Assertion architecture

- Immediate assertions: ``$display``
+ ``if`` checks in procedural TB
(baseline)
- Concurrent-style patterns:
Properties described in examples
where SVA subset supported
- Assertion libraries: Reusable
check macros/modules in
``assertion_libraries/`` examples
- Failure action: ``$error`` /
``fatal`` or abort simulation with
non-zero status

Verification Architecture

Diagram: Verification Architecture
(render with mmdc when available)
flowchart LR
STIM[reset + enable tests] --> SIM[vvp / verilator run]
SIM --> AST[assert on posedge clk]
AST --> COV[coverage bins optional]
COV --> CLOSE[closure report]

3. Coverage model architecture

- Functional coverage points:
 - User-defined bins on opcodes, states, or cross terms
- Code coverage: Line/branch awareness via simulator reports where available
- Feedback loop: Coverage holes drive new directed or random tests (documented in guides)

Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TD

BASIC[basic_assertions] --> LIB[assertion_libraries]

LIB --> COV[coverage_analysis]

COV --> GAP[find coverage holes]

GAP --> REG[module7.sh --check]

RTL block diagram (reference)

Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

DUT[counter_4bit] --> IMM[immediate checks]

DUT --> CLKAST[clocked assertions]

IMM --> LOG[assert pass/fail count]

CLKAST --> LOG

LIB[assertion library macros] --> CLKAST

Module 7: DUT hierarchy and signal flow.

Verification / testbench diagram (reference)

Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart LR

STIM[reset + enable tests] --> SIM[vvp / verilator run]

SIM --> AST[assert on posedge clk]

AST --> COV[coverage bins optional]

COV --> CLOSE[closure report]

Module 7: stimulus, observation, and checking.

Assertions Makefile — compile DUT+TB

```
basic_assertions_verilog: basic_assertions_verilog.v
    @echo "=== Compiling Verilog assertions example ==="
    $(IVERILOG) -g2012 -o basic_assertions
basic_assertions_verilog.v $(DUT_DIR)/counters/counter_4bit.v
    @echo "=== Running simulation ==="
    $(VVP) basic_assertions
```

Clocked assertion check

```
// Clock generation
initial begin
    clk = 0;
    forever #10 clk = ~clk;
end

// Assertion: Reset should clear counter
always @(posedge clk) begin
    if (!rst_n) begin
        assertion_count = assertion_count + 1;
        if (count == 4'b0000) begin
            assertion_pass = assertion_pass + 1;
            $display("[ASSERT PASS] Reset assertion: count is 0");
        end
    end else begin
        assertion_fail = assertion_fail + 1;
        $display("[ASSERT FAIL] Reset assertion: count should be 0");
    end
end
```

DUT under assertion test

```
// Instantiate DUT
counter_4bit dut (
    .clk(clk),
    .rst_n(rst_n),
    .en(en),
    .count(count)
);
```

Execution & simulation flow

How the example runs (toolchain)

- Makefile: Verilator compiles RTL + SystemVerilog testbench into a C++ model
- sim_main.cpp: generates clk/rst_n, calls eval() until \$finish
- Directed test (initial block or C++): drive stimulus, wait for DUT flags
- Self-check: compare outputs; print PASS/FAIL (see terminal demo slide)
- Repo path: module7/examples/<example>/ — run make run from that directory

Directed test execution sequence (1)

- Clocked assertions on posedge
- summary block before \$finish
- non-zero fail triggers debug
- Embed checks in TB
- run simulation

Directed test execution sequence (2)

- collect assertion_pass/fail
- analyze coverage gaps
- `./scripts/module7.sh --check`

Run assertion example

```
basic_assertions_verilog: basic_assertions_verilog.v
    @echo "=== Compiling Verilog assertions example ==="
    $(IVERILOG) -g2012 -o basic_assertions
basic_assertions_verilog.v $(DUT_DIR)/counters/counter_4bit.v
    @echo "=== Running simulation ==="
    $(VVP) basic_assertions
```

Reset assertion property

```
// Assertion: Reset should clear counter
always @(posedge clk) begin
    if (!rst_n) begin
        assertion_count = assertion_count + 1;
        if (count == 4'b0000) begin
            assertion_pass = assertion_pass + 1;
            $display("[ASSERT PASS] Reset assertion: count is 0
when
rst_n=0");
        end else begin
            assertion_fail = assertion_fail + 1;
            $display("[ASSERT FAIL] Reset assertion: count should
be
0 when rst_n=0, got %0d", count);
        end
    end
end
end
```

Verification & testing methods

1. Assertion-based verification (ABV)

- Property targets: Reset behavior, one-hot select, counter monotonicity, no X on outputs
- Assume/guarantee mindset: Environment assumptions vs DUT guarantees in examples
- Debug: Assertion name and time stamp in failure message — rerun with VCD at failure cycle

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
    BASIC[basic_assertions] --> LIB[assertion_libraries]
    LIB --> COV[coverage_analysis]
    COV --> GAP[find coverage holes]
    GAP --> REG[module7.sh --check]
```

2. Coverage-driven closure

- Goals: Target 100% on critical crosses; accept waived bins with documented rationale
- Sampling: Coverpoints updated each transaction or clock in TB code
- Reports: Text summaries from ``coverage_analysis`` examples — gap list for next tests

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
    BASIC[basic_assertions] --> LIB[assertion_libraries]
    LIB --> COV[coverage_analysis]
    COV --> GAP[find coverage holes]
    GAP --> REG[module7.sh --check]
```

3. Metrics and sign-off prep

- Pass/fail plus coverage: Exit criteria require zero assertion failures and met coverage goals
- Regression: Re-run assertion suite after RTL/TB edits —
`./scripts/module7.sh --check`
- Bridge to Module 8: Metrics feed verification plan and sign-off checklist

Testing Methods

```
Diagram: Testing Methods
(render with mmdc when available)
flowchart TD
    BASIC[basic_assertions] --> LIB[assertion_libraries]
    LIB --> COV[coverage_analysis]
    COV --> GAP[find coverage holes]
    GAP --> REG[module7.sh --check]
```


Assertion library module

```
/**
 * Assertion Library Example - Verilog
 *
 * Demonstrates:
 * - Reusable assertion functions/tasks
 * - Assertion organization
 * - Assertion library patterns
 *
 * Usage:
 *   iverilog -g2012 -o assertion_lib assertion_lib_verilog.v
 *   ../dut/simple_gates/and_gate.v
 *   vvp assertion_lib
 */
```

```
`timescale 1ns/1ps
```

```
// Assertion library
```

Coverage analysis workflow

```
# Makefile for coverage analysis examples
# Note: Coverage analysis examples are coming soon

all:
    @echo "===== "
    @echo "Coverage Analysis Examples"
    @echo "===== "
    @echo "Coverage analysis examples are coming soon."
    @echo "See module7/README.md for details."
    @echo "===== "

clean:
    @echo "No build artifacts to clean"

.PHONY: all clean
```

Assertion summary before \$finish

```
// Print assertion summary
    $display("");
    $display("=====");
    $display("Assertion Summary");
    $display("=====");
    $display("Total assertions:  %0d", assertion_count);
    $display("Passed:           %0d", assertion_pass);
    $display("Failed:             %0d", assertion_fail);
    $display("=====");

    if (assertion_fail == 0) begin
        $display("✓ All assertions PASSED!");
    end else begin
        $display("✗ Some assertions FAILED!");
    end
end
```

Syllabus topics

1. Coverage Fundamentals (1/5)

- What is Coverage?
- Verification completeness
- Coverage metrics
- Coverage goals
- Coverage-driven verification
- Coverage Types (Code, Functional, Toggle)

1. Coverage Fundamentals (2/5)

- Code coverage
- Functional coverage
- Toggle coverage
- Coverage relationships
- Coverage Metrics
- Coverage percentage

1. Coverage Fundamentals (3/5)

- Coverage bins
- Coverage goals
- Coverage closure
- Coverage Goals
- Setting coverage targets
- Coverage planning

1. Coverage Fundamentals (4/5)

- Coverage achievement
- Coverage sign-off
- Coverage-Driven Verification
- Using coverage to guide testing
- Coverage-directed test generation
- Coverage optimization

1. Coverage Fundamentals (5/5)

- Coverage best practices

2. Code Coverage (1/6)

- Line Coverage
- Line execution
- Line coverage metrics
- Line coverage analysis
- Branch Coverage
- Branch execution

2. Code Coverage (2/6)

- Branch coverage metrics
- Branch coverage analysis
- Condition Coverage
- Condition execution
- Condition coverage metrics
- Condition coverage analysis

2. Code Coverage (3/6)

- Path Coverage
- Path execution
- Path coverage metrics
- Path coverage analysis
- iverilog Coverage Capabilities
- Built-in coverage

2. Code Coverage (4/6)

- Coverage limitations
- Coverage tools
- Coverage analysis
- Verilator Coverage Capabilities (--coverage)
- Coverage flag
- Coverage generation

2. Code Coverage (5/6)

- Coverage data
- Coverage analysis
- Coverage Tools and Analysis
- Coverage tools
- Coverage analysis
- Coverage reporting

2. Code Coverage (6/6)

- Coverage best practices
- ``code_coverage_iverilog.v``: Manual code coverage tracking with iverilog
- ``code_coverage_verilator.cpp``: Code coverage with Verilator (--coverage flag)

3. Functional Coverage (Manual Implementation)

(1/7)

- Functional Coverage Concepts
- Functional coverage definition
- Coverage bins
- Coverage goals
- Coverage metrics
- Manual Functional Coverage Using Counters

3. Functional Coverage (Manual Implementation)

(2/7)

- Counter-based tracking
- Coverage bin implementation
- Coverage collection
- Coverage analysis
- Coverage Bins Implementation (Verilog and C++)
- Bin definition

3. Functional Coverage (Manual Implementation)

(3/7)

- Bin tracking
- Bin coverage
- Bin analysis
- Cross Coverage Using Manual Tracking
- Cross coverage definition
- Cross coverage tracking

3. Functional Coverage (Manual Implementation)

(4/7)

- Cross coverage analysis
- Cross coverage best practices
- Coverage Collection and Analysis
- Coverage collection
- Coverage storage
- Coverage analysis

3. Functional Coverage (Manual Implementation)

(5/7)

- Coverage reporting
- Verilog: Using `$display/$monitor` for Coverage Tracking
- Display-based tracking
- Monitor-based tracking
- Coverage logging
- Coverage analysis

3. Functional Coverage (Manual Implementation) (6/7)

- C++: Using Variables and Data Structures for Coverage Tracking
 - Variable-based tracking
 - Data structure tracking
 - Coverage storage
 - Coverage analysis
- ``functional_coverage_verilog.v``: Manual functional coverage using counters

3. Functional Coverage (Manual Implementation) (7/7)

- ``functional_coverage_cpp.cpp``: Functional coverage using C++ data structures

4. Toggle Coverage (1/4)

- Signal Toggling
- Toggle definition
- Toggle tracking
- Toggle metrics
- Toggle analysis
- Toggle Coverage Metrics

4. Toggle Coverage (2/4)

- Toggle percentage
- Toggle goals
- Toggle closure
- Toggle best practices
- Toggle Analysis
- Toggle identification

4. Toggle Coverage (3/4)

- Toggle gaps
- Toggle optimization
- Toggle best practices
- Coverage Reporting
- Toggle reports
- Toggle visualization

4. Toggle Coverage (4/4)

- Toggle documentation
- Toggle best practices

5. Basic Assertion Concepts (1/7)

- What are Assertions?
- Assertion definition
- Assertion purpose
- Assertion benefits
- Assertion types
- Assertion Purpose and Benefits

5. Basic Assertion Concepts (2/7)

- Design verification
- Bug detection
- Documentation
- Debugging aid
- Simple Assertion Patterns
- Equality assertions

5. Basic Assertion Concepts (3/7)

- Range assertions
- Boolean assertions
- Timing assertions
- Verilog: Using if-else for Assertions
- If-else patterns
- Assertion implementation

5. Basic Assertion Concepts (4/7)

- Error reporting
- Assertion best practices
- Verilog: Using `$assert` (if supported)
- `$assert` syntax
- `$assert` usage
- `$assert` limitations

5. Basic Assertion Concepts (5/7)

- \$assert best practices
- C++: Using assert() Macro
- assert() syntax
- assert() usage
- assert() limitations
- assert() best practices

5. Basic Assertion Concepts (6/7)

- C++: Using Custom Assertion Functions
- Custom function design
- Custom function implementation
- Custom function usage
- Custom function best practices
- Error Reporting in Assertions

5. Basic Assertion Concepts (7/7)

- Error messages
- Error logging
- Error handling
- Error best practices
- ``basic_assertions_verilog.v``: Basic assertions using if-else patterns
- ``basic_assertions_cpp.cpp``: Basic assertions using `assert()` and custom functions

6. Assertion Implementation Patterns (1/7)

- Clock-Based Assertions
- Clock synchronization
- Clock-based checks
- Clock assertion patterns
- Clock assertion best practices
- Reset Assertions

6. Assertion Implementation Patterns (2/7)

- Reset checks
- Reset assertion patterns
- Reset assertion timing
- Reset assertion best practices
- Data Validity Assertions
- Data range checks

6. Assertion Implementation Patterns (3/7)

- Data validity patterns
- Data assertion implementation
- Data assertion best practices
- Protocol Assertions (Basic Patterns)
- Protocol checks
- Protocol assertion patterns

6. Assertion Implementation Patterns (4/7)

- Protocol assertion implementation
- Protocol assertion best practices
- Assertion Organization
- Assertion placement
- Assertion grouping
- Assertion hierarchy

6. Assertion Implementation Patterns (5/7)

- Assertion organization best practices
- Reusable Assertion Functions/Tasks
- Assertion library design
- Assertion library implementation
- Assertion library usage
- Assertion library best practices

6. Assertion Implementation Patterns (6/7)

- Verilog: Assertion Tasks and Functions
 - Task-based assertions
 - Function-based assertions
 - Assertion organization
 - Assertion best practices
- C++: Assertion Helper Functions and Classes

6. Assertion Implementation Patterns (7/7)

- Function-based assertions
- Class-based assertions
- Assertion organization
- Assertion best practices
- ``assertion_lib_verilog.v``: Reusable assertion library in Verilog
- ``assertion_lib_cpp.cpp``: Reusable assertion library in C++

7. Coverage Analysis (1/5)

- Coverage Collection
- Coverage data collection
- Coverage storage
- Coverage management
- Coverage best practices
- Coverage Reporting

7. Coverage Analysis (2/5)

- Coverage report generation
- Coverage report format
- Coverage report analysis
- Coverage report best practices
- Coverage Analysis
- Coverage gap identification

7. Coverage Analysis (3/5)

- Coverage analysis methods
- Coverage optimization
- Coverage best practices
- Coverage Gaps Identification
- Gap identification
- Gap analysis

7. Coverage Analysis (4/5)

- Gap closure
- Gap best practices
- Coverage Closure Strategies
- Closure planning
- Closure methods
- Closure achievement

7. Coverage Analysis (5/5)

- Closure best practices

8. Coverage-Driven Test Generation (1/6)

- Using Coverage to Guide Testing
- Coverage-directed testing
- Coverage-based test selection
- Coverage optimization
- Coverage best practices
- Coverage-Directed Test Generation

8. Coverage-Driven Test Generation (2/6)

- Test generation methods
- Test generation strategies
- Test generation optimization
- Test generation best practices
- Coverage Optimization
- Coverage improvement

8. Coverage-Driven Test Generation (3/6)

- Coverage optimization methods
- Coverage optimization strategies
- Coverage optimization best practices
- Test Selection Based on Coverage
- Test selection methods
- Test selection strategies

8. Coverage-Driven Test Generation (4/6)

- Test selection optimization
- Test selection best practices
- Location: ``module7/examples/code_coverage/``
- Demonstrates: Code coverage tracking and analysis
- Location: ``module7/examples/functional_coverage/``
- Demonstrates: Manual functional coverage implementation

8. Coverage-Driven Test Generation (5/6)

- Location: ``module7/examples/assertions/``
- Demonstrates: Basic assertion patterns and implementation
- Location: See coverage examples
- Demonstrates: Coverage-driven test generation
- Location: ``module7/examples/coverage_analysis/``
(coming soon)
- Demonstrates: Coverage analysis and closure

8. Coverage-Driven Test Generation (6/6)

- Location: ``module7/examples/assertion_libraries/``
- Demonstrates: Reusable assertion libraries

Hands-on examples

Module 7 self-check

```
$ ./scripts/module7.sh --help
```

Terminal: module7_check

Usage: ./scripts/module7.sh [OPTIONS]

Module 7: Coverage and Assertions

This script runs examples and tests for Module 7.

Focuses on basic coverage and assertions that work with both tools.

OPTIONS:

Examples:

--code-coverage	Run code coverage examples
--functional-coverage	Run functional coverage examples
--assertions	Run assertion examples
--coverage-analysis	Run coverage analysis examples
--assertion-libraries	Run assertion library examples
--all-examples	Run all examples (default)
--skip-examples	Skip all examples

Tests:

--verilog-tests	Run Verilog testbenches
--cpp-tests	Run C++ testbenches
--all-tests	Run all tests (default)
--skip-tests	Skip all tests

Environment:

--jobs N	Number of parallel build jobs (default: 8)
--no-clean	Skip cleaning before build (faster rebuilds)

Other:

--help, -h	Show this help message
------------	------------------------

EXAMPLES:

```
# Run all examples and tests
./scripts/module7.sh
```

```
# Run only code coverage examples
./scripts/module7.sh --code-coverage
```

Demo: Assertions

```
$ cd module7/examples/assertions && make all
```

Terminal: ex01_assertions

```
=== Compiling Verilog assertions example ===
iverilog -g2012 -o basic_assertions basic_assertions_verilog.v ../dut/counters/counter_4bit.v
=== Running simulation ===
vvp basic_assertions
=====
Basic Assertions Example
=====
VCD info: dumpfile basic_assertions.vcd opened for output.
[ASSERT FAIL] Reset assertion: count should be 0 when rst_n=0, got x

Test 1: Reset assertion
[ASSERT PASS] Reset assertion: count is 0 when rst_n=0
[ASSERT PASS] Reset assertion: count is 0 when rst_n=0

Test 2: Increment assertion
[ASSERT FAIL] Increment assertion: count should increment, prev=0, curr=0

Test 3: Range assertion

=====
Assertion Summary
=====
Total assertions: 33
Passed:          31
Failed:           2
=====
X Some assertions FAILED!
=====
basic_assertions_verilog.v:135: $finish called at 375000 (1ps)
=== Compiling C++ assertions example ===
verilator --cc --exe --build -I../dut/counters \
[]../dut/counters/counter_4bit.v basic_assertions_cpp.cpp
make: verilator: No such file or directory
make: *** [Makefile:18: basic_assertions_cpp] Error 127
```

Demo: Assertion libraries

```
$ cd module7/examples/assertion_libraries && make all
```

Terminal: ex02_assertion_libraries

```
=== Compiling Verilog assertion library example ===
iverilog -g2012 -o assertion_lib assertion_lib_verilog.v ../dut/simple_gates/and_gate.v
=== Running simulation ===
vvp assertion_lib
=====
Assertion Library Example
=====
[ASSERT PASS]      AND(0,0) should be 0
[ASSERT PASS]      AND(0,1) should be 0
[ASSERT PASS]      AND(1,0) should be 0
[ASSERT PASS]      AND(1,1) should be 1
[ASSERT PASS]      Output should be in range [0:1]
[ASSERT PASS]      Output should be boolean
=====
Assertion library demonstration complete
=====
assertion_lib_verilog.v:115: $finish called at 30000 (1ps)
=== Compiling C++ assertion library example ===
verilator --cc --exe --build -I../dut/simple_gates \
[]../dut/simple_gates/and_gate.v assertion_lib_cpp.cpp
make: verilator: No such file or directory
make: *** [Makefile:18: assertion_lib_cpp] Error 127
```

Demo: Coverage analysis

```
$ ls -la module7/examples/coverage_analysis
```

Terminal: ex03_coverage_analysis

```
total 0
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 .
drwxrwxrwx 1 yongfu yongfu 4096 May 28 12:19 ..
-rwxrwxrwx 1 yongfu yongfu 446 May 28 12:19 Makefile
```


Practice & assessment

What you should know (1/8)

- Understand fundamental coverage concepts
- Measure code coverage with both tools
- Implement manual functional coverage (both paradigms)
- Write basic assertions in Verilog and C++
- Organize assertions effectively
- Analyze coverage reports

What you should know (2/8)

- Use coverage to guide verification
- Achieve basic coverage closure
- Apply assertion-based verification patterns
- Use tool-specific coverage flags
- Track coverage manually
- Analyze coverage reports

What you should know (3/8)

- Create coverage bins
- Track coverage manually
- Generate coverage reports
- Use C++ data structures
- Track coverage manually
- Generate coverage reports

What you should know (4/8)

- Use if-else patterns
- Create assertion tasks
- Organize assertions
- Use assert() macro
- Create custom assertion functions
- Organize assertions

What you should know (5/8)

- Design reusable assertions
- Organize assertion library
- Use assertion library
- Identify coverage gaps
- Plan coverage closure
- Achieve coverage goals

What you should know (6/8)

- Use coverage to guide testing
- Generate tests based on coverage
- Optimize test selection
- Can understand fundamental coverage concepts
- Can measure code coverage with both tools
- Can implement manual functional coverage (both paradigms)

What you should know (7/8)

- Can write basic assertions in Verilog and C++
- Can organize assertions effectively
- Can analyze coverage reports
- Can use coverage to guide verification
- Can achieve basic coverage closure
- Can apply assertion-based verification patterns

What you should know (8/8)

- Module 8: Verification Methodology and Best Practices - Learn industry best practices and verificat...
- Icarus Verilog Documentation:
<http://iverilog.wikia.com/>
- Verilator Documentation: <https://verilator.org/>
- GTKWave Documentation:
<http://gtkwave.sourceforge.net/>
- IEEE 1364-2005 Standard: Verilog Hardware Description Language

Assessment checklist

- Can understand fundamental coverage concepts
- Can measure code coverage with both tools
- Can implement manual functional coverage (both paradigms)
- Can write basic assertions in Verilog and C++
- Can organize assertions effectively
- Can analyze coverage reports

Summary & next steps

- Master basic coverage analysis and assertion-based verification without SystemVerilog dependencies
- Complete module7/CHECKLIST.md
- Review module7/EXAMPLES.md and run each lab
- Next: Next module in course